

📁 Guia dos Arquivos do AlunosApp

Cada arquivo do projeto tem uma função específica. Entender quem faz o quê é o primeiro passo para não se perder quando algo dá errado. Este guia mostra, um por um, o papel de cada peça.

📁 Onde cada arquivo mora

Todo o app vive dentro da pasta `www/`. Fora dela, só existe o workflow do GitHub. Essa separação é o que permite que o **mesmo código** seja usado como site (PWA) e como app (APK).

```
seu-repo/
├── .github/
│   ├── workflows/
│   └── build.yml ← o robô do GitHub
└──
    └── www/ ← todo o app vive aqui
        ├── index.html ← a estrutura da tela
        ├── style.css ← a aparência
        ├── app.js ← o cérebro (CRUD)
        ├── manifest.json ← a identidade do PWA
        ├── sw.js ← o cache offline
        └── icons/
            ├── icon-192.png ← ícone pequeno
            └── icon-512.png ← ícone grande
```



index.html

MARCAÇÃO

📌 `www/index.html`

O QUE É

É o **esqueleto da tela**. Contém a estrutura visual: título, campos de digitação (Nome, E-mail, Idade), botão "Salvar Aluno" e a tabela onde os alunos aparecem.

PARA QUE SERVE

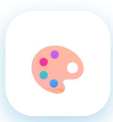
Sem ele, o navegador não tem o que mostrar. Toda página web começa por um HTML — é o ponto de entrada do app.

SE FALTAR OU ESTIVER ERRADO

O site abre em branco. O APK também — porque o Capacitor procura por esse arquivo para carregar dentro do app Android.

💡 ANALOGIA

Imagine uma casa: o `index.html` é a **planta baixa** — mostra onde estão as paredes, portas e janelas, mas não define a cor nem o acabamento.



style.css

ESTILO

📍 `www/style.css`

O QUE É

É a **aparência** do app. Define cores, fontes, tamanhos, espaçamentos, sombras, tudo que deixa a interface bonita.

PARA QUE SERVE

Deixa o app com aparência de aplicativo nativo — fundo escuro, botões grandes, tabela estilizada. No Android, o app respeita as bordas arredondadas do sistema.

SE FALTAR OU ESTIVER ERRADO

O app continua funcionando, mas fica **feio**: tudo preto no branco, botões sem cor, sem espaçamento. Parece site dos anos 90.

💡 ANALOGIA

Se o HTML é a planta da casa, o CSS é a **pintura, os móveis e o acabamento**. A casa funciona sem eles, mas ninguém quer morar.



app.js

LÓGICA

📌 `www/app.js`

O QUE É

É o **cérebro do app**. Contém a lógica do CRUD: pegar o que o usuário digitou, salvar no `LocalStorage`, listar na tabela e apagar quando clica em "Excluir".

PARA QUE SERVE

Sem ele, os botões não fazem nada. Você digita o nome, clica em Salvar, e nada acontece — porque ninguém está escutando os cliques nem manipulando os dados.

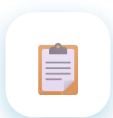
SE FALTAR OU ESTIVER ERRADO

Sintomas clássicos:

- Botão Salvar não faz nada.
- Tabela mostra `${a.id}` e `${a.nome}` em vez dos valores.
- Dados não persistem quando fecha o app.

💡 ANALOGIA

Numa cozinha, o HTML é a bancada, o CSS é a decoração, e o `app.js` é o **cozinheiro** — ele pega os ingredientes (inputs), mistura (lógica) e serve o prato (tabela).



manifest.json

IDENTIDADE PWA

📌 `www/manifest.json`

O QUE É

É a **carteira de identidade** do app para o navegador. Contém o nome do app, o nome curto que aparece embaixo do ícone, a cor da barra de status, e a lista de ícones.

PARA QUE SERVE

Faz o Chrome oferecer "*Adicionar à tela inicial*". Quando instalado, o app abre em **tela cheia**, sem barra de endereço — parece nativo.

SE FALTAR OU ESTIVER ERRADO

O site funciona no navegador normalmente, mas **o aviso de instalação nunca aparece**. O aluno não consegue "instalar" como se fosse um app.

💡 ANALOGIA

É como o **RG do aplicativo** — diz quem ele é, como se chama, e que ícone usar quando aparecer na tela do celular.

**sw.js****SERVICE WORKER**

www/sw.js

O QUE É

É o arquivo que roda **em segundo plano**, separado da página. Ele guarda uma cópia dos arquivos do app no celular, para o app funcionar **offline**.

PARA QUE SERVE

Sem internet, o app abre do cache e continua funcionando com o LocalStorage. Também é um dos requisitos para o Chrome considerar o site "instalável" como PWA.

SE FALTAR OU ESTIVER ERRADO

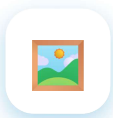
- O app não abre offline.
- O Chrome para de oferecer instalação.
- O celular começa a mostrar **versões antigas** do app mesmo depois de atualizar (problema de cache).

💡 ANALOGIA

É como uma **caixa térmica de comida**: guarda porções do app no celular. Quando você não pode ir ao restaurante (sem internet), abre a caixa e come do que já tem guardado.

⚠️ CUIDADO IMPORTANTE

Toda vez que você **editar qualquer arquivo do app**, precisa trocar `alunosapp-v1` para `alunosapp-v2`, `-v3`, etc. dentro do `sw.js`. Se não fizer isso, o celular continua mostrando a versão antiga.



icon-192.png e icon-512.png

ÍCONES

📍 www/icons/

O QUE É

São as **imagens do ícone** do aplicativo, em dois tamanhos diferentes: 192×192 pixels e 512×512 pixels. Ambas são exigidas pelo `manifest.json`.

PARA QUE SERVE

É o que aparece na **tela inicial do celular** quando o app é instalado, e na lista de aplicativos recentes do Android. Sem eles, o Chrome nem permite instalar a PWA.

SE FALTAR OU ESTIVER ERRADO

O aviso "Adicionar à tela inicial" não aparece no Chrome. E se o app for instalado de outra forma, o ícone aparece quebrado (quadrado cinza).

💡 ANALOGIA

É a **foto de perfil** do aplicativo. É o que as pessoas veem quando procuram o app no celular. Pode ter tamanho pequeno (ícone normal) e grande (tela de detalhes).



build.yml

AUTOMAÇÃO

📍 `.github/workflows/build.yml`

O QUE É

É a **receita de bolo** que o GitHub segue automaticamente. Ele diz: "sempre que houver um push na branch `main`, publique o site no Pages e compile o APK do Android".

PARA QUE SERVE

Permite gerar o APK **sem instalar nada no computador**. O GitHub aluga um servidor na nuvem, executa os comandos do Capacitor e devolve o arquivo pronto para download.

SE FALTAR OU ESTIVER ERRADO

- Nenhum site é publicado no GitHub Pages.

- Nenhum APK é gerado.
- A aba Actions mostra erro ou não mostra nada.

💡 ANALOGIA

É o **robô de linha de montagem**. Você entrega a matéria-prima (os arquivos do app), ele segue o passo a passo e devolve o produto embalado (site + APK).

⚠️ CUIDADO IMPORTANTE

Este arquivo **não pode ter tabs**, só espaços. E o caminho tem que ser exatamente `.github/workflows/build.yml` — qualquer variação e o GitHub ignora.

🔄 Como os arquivos conversam entre si

Quando o aluno abre o app no celular, acontece esta sequência:



Perceba: **todos os arquivos trabalham em conjunto**. Se um falha, o app fica capenga em alguma parte. É por isso que todos os 7 precisam estar no lugar certo.

🎯 Resumo de uma frase por arquivo



`index.html`

O esqueleto — mostra a estrutura da tela.



`style.css`

A aparência — cores, fontes, espaçamento.



`app.js`

O cérebro — faz o CRUD funcionar.



`manifest.json`

A identidade — permite instalar como app.

**sw.js**

O cache — faz funcionar offline.

**icon-192 / icon-512**

Os ícones — aparecem na tela inicial.

**build.yml**

O robô — publica o site e gera o APK.

**Próximos passos**

Agora que você já entende o papel de cada arquivo, o próximo passo é **colocar a mão na massa**. Consulte o Manual Completo para ver a ordem exata de criação dos arquivos, cópia de códigos, geração de ícones e publicação via GitHub Actions.

Lembre-se: **todos os arquivos precisam existir**. Um projeto com 5 dos 7 arquivos pode até abrir, mas vai falhar em algum momento — ou na instalação como PWA, ou na geração do APK, ou em ambos.